STUDENT PERFORMANCE ANALYSIS

Exploratory Data Analysis EDA Mini Project

Project overview

This project uses Exploratory Data Analysis EDA to examine how selected student-related factors are associated with final mathematics grades. The analysis focuses on study time, absences, past class failures, school support, age, and sex.

Objective

To inspect the dataset, check its basic quality, summarize final-grade patterns, compare average final grades across selected groups, and visualize the main relationships.

Dataset

The notebook reads student-mat.csv using a semicolon ; as the separator. The loaded dataset contains 395 rows and 33 columns.

Tools and libraries

Pandas: loading, inspecting, grouping, and summarizing tabular data.

NumPy: numerical computing support.

Matplotlib: creating and formatting plots.

Seaborn: statistical visualizations.

Note: This is an exploratory analysis. The comparisons show patterns in this dataset and do not prove that any factor causes a change in grades.

1. Import the libraries

I imported NumPy, Pandas, Matplotlib, and Seaborn. These libraries support data handling, numerical work, and visualizing patterns in the dataset.

python

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

2. Load the dataset

I loaded student-mat.csv into a Pandas DataFrame named data. The sep=";" argument is important because the values in this CSV file are separated by semicolons rather than commas. Displaying data lets me inspect the loaded table.

python

```
data=pd.read_csv("student-mat.csv",sep=";")
data
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3
...	...	...	...	...	...	...	...	...	...	...	...	...	...
390	MS	M	20	U	LE3	A	2	2	services	services	...	5	5
391	MS	M	17	U	LE3	T	3	1	services	services	...	2	4
392	MS	M	21	R	GT3	T	1	1	other	other	...	5	5
393	MS	M	18	R	LE3	T	3	2	services	other	...	4	4
394	MS	M	19	U	LE3	T	1	1	other	at_home	...	3	2

395 rows × 33 columns

3. Preview the first rows

I used head to view the first five rows. This helps me understand how the records and column values are organized.

python

```
data.head()
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goo
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	

5 rows × 33 columns

4. Inspect the column names

I displayed data.columns to see the available variables and confirm which fields could be useful for the analysis.

python

```
data.columns
```

```
Index(['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu',
'Fedu',
       'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime',
       'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery',
       'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout',
'Dalc',
       'Walc', 'health', 'absences', 'G1', 'G2', 'G3'],
      dtype='str')
```

5. Preview the data again

This repeats the first-row preview. It provides another look at the records, although it is not essential because the same preview was already shown above.

python

```
data.head()
```

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	famrel	freetime	goc
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	4	3	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	5	3	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	4	3	
3	GP	F	15	U	GT3	T	4	2	health	services	...	3	2	
4	GP	F	16	U	GT3	T	3	3	other	other	...	4	3	

5 rows × 33 columns

6. Check the dataset dimensions

I used shape to find the number of rows and columns. The result, $395, 33$, means the dataset has 395 student records and 33 columns.

python

```
data.shape
```

```
(395, 33)
```

7. Check column data types and non-null counts

I used info to review the column names, data types, and number of non-null values. This gives an overview of the dataset structure and helps identify columns that may need further inspection.

python

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 395 entries, 0 to 394
Data columns (total 33 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   school      395 non-null    str
 1   sex         395 non-null    str
 2   age         395 non-null    int64
 3   address     395 non-null    str
 4   famsize     395 non-null    str
 5   Pstatus     395 non-null    str
 6   Medu        395 non-null    int64
 7   Fedu        395 non-null    int64
 8   Mjob        395 non-null    str
 9   Fjob        395 non-null    str
 10  reason      395 non-null    str
 11  guardian    395 non-null    str
 12  traveltime  395 non-null    int64
 13  studytime   395 non-null    int64
 14  failures    395 non-null    int64
 15  schoolsup   395 non-null    str
 16  famsup      395 non-null    str
 17  paid        395 non-null    str
 18  activities  395 non-null    str
 19  nursery     395 non-null    str
 20  higher      395 non-null    str
 21  internet    395 non-null    str
 22  romantic    395 non-null    str
 23  famrel      395 non-null    int64
 24  freetime    395 non-null    int64
 25  goout       395 non-null    int64
 26  Dalc        395 non-null    int64
 27  Walc        395 non-null    int64
 28  health      395 non-null    int64
 29  absences    395 non-null    int64
 30  G1          395 non-null    int64
 31  G2          395 non-null    int64
 32  G3          395 non-null    int64
dtypes: int64(16), str(17)
memory usage: 102.0 KB
```

8. Check for missing values

I counted missing values in every column with isnull.sum. In the saved output, every column has a count of zero, so no missing values were detected by this check.

python

```python
data.isnull().sum()
```

```
school        0
sex           0
age           0
address       0
famsize       0
Pstatus       0
Medu          0
Fedu          0
Mjob          0
Fjob          0
reason        0
guardian      0
traveltime    0
studytime     0
failures      0
schoolsup     0
famsup        0
paid          0
activities    0
nursery       0
higher        0
internet      0
romantic      0
famrel        0
freetime      0
goout         0
Dalc          0
Walc          0
health        0
absences      0
G1            0
G2            0
G3            0
dtype: int64
```

9. Check for duplicate rows

I used duplicated.sum to count rows that exactly repeat an earlier row. The saved output is 0, meaning no exact duplicate rows were detected in the original dataset.

python

```
data.duplicated().sum()
```

```
np.int64(0)
```

10. Summarize the grade columns

I used describe on G1, G2, and G3, representing the first-period, second-period, and final grades. The summary includes the count, mean, standard deviation, minimum, quartiles, and maximum, helping me understand the grade ranges and distributions.

python

```
data[["G1","G2","G3"]].describe()
```

	G1	G2	G3
count	395.000000	395.000000	395.000000
mean	10.908861	10.713924	10.415190
std	3.319195	3.761505	4.581443
min	3.000000	0.000000	0.000000
25%	8.000000	9.000000	8.000000
50%	11.000000	11.000000	11.000000
75%	13.000000	13.000000	14.000000
max	19.000000	19.000000	20.000000

11. Study time and final grade

I grouped students by studytime and calculated the average final grade $G3$ in each group. This helps compare final-grade averages across the dataset's study-time categories. The studytime values are coded categories, so they should be interpreted using the dataset's coding scheme.

python

```python
data.groupby("studytime")["G3"].mean()
```

```
studytime
1    10.047619
2    10.171717
3    11.400000
4    11.259259
Name: G3, dtype: float64
```

12. Inspect study-time values

I displayed the studytime column to inspect the values recorded for students. This is a direct column preview rather than a summary or visualization.

python

```python
data["studytime"]
```

```
0      2
1      2
2      2
3      3
4      2
      ..
390    2
391    1
392    1
393    1
394    1
Name: studytime, Length: 395, dtype: int64
```

13. Absences and final grade

I grouped students by their number of absences and calculated the mean G3 for each absence count. This explores whether average final grades differ across absence levels. Groups with very few students can have unstable averages, so individual values should be interpreted carefully.

python

```python
data.groupby("absences")["G3"].mean()
```

```
absences
0      8.443478
1     13.000000
2     11.907692
3     11.500000
4     11.283019
5     11.600000
6     11.774194
7     11.857143
8     10.590909
9     13.333333
10    11.235294
11    11.666667
12    10.750000
13    12.666667
14     9.333333
15     8.333333
16     9.857143
17    10.000000
18     8.400000
19    10.000000
20    11.500000
21    18.000000
22     8.000000
23    13.000000
24    18.000000
25    11.000000
26     6.000000
28     9.000000
30     8.000000
38     8.000000
40    11.000000
54    11.000000
56     8.000000
75     9.000000
Name: G3, dtype: float64
```

14. Past failures and final grade

I grouped students by the number of past class failures and calculated their average final grade. In the saved results, the average G3 is lower in the categories with more past failures; this is an observed association in this dataset, not proof of cause and effect.

python

```
data.groupby("failures")["G3"].mean()
```

```
failures
0    11.253205
1     8.120000
2     6.235294
3     5.687500
Name: G3, dtype: float64
```

15. Inspect past-failure values

I displayed the failures column to inspect the recorded categories. This is a direct view of the variable and complements the grouped average calculated above.

python

```
data["failures"]
```

```
0      0
1      0
2      3
3      0
4      0
      ..
390    2
391    0
392    3
393    0
394    0
Name: failures, Length: 395, dtype: int64
```

16. Extra school support and final grade

I compared average final grades for students recorded as receiving extra educational support $schoolsup = yes$ and those not receiving it $schoolsup = no$. The averages describe the groups in this dataset; they do not show whether support itself caused the difference, because students may receive support for different reasons.

python

```python
data.groupby("schoolsup")["G3"].mean()
```

```
schoolsup
no     10.561047
yes     9.431373
Name: G3, dtype: float64
```

17. Age and final grade

I grouped students by age and calculated the mean final grade for each age. This allows a comparison of grade averages across age groups. Some age categories may contain fewer students than others.

python

```python
data.groupby("age")["G3"].mean()
```

```
age
15    11.256098
16    11.028846
17    10.275510
18     9.548780
19     8.208333
20    14.000000
21     7.000000
22     8.000000
Name: G3, dtype: float64
```

19. Sex and final grade

I grouped students by the sex category and calculated the average final grade for each group. This compares group averages in the dataset and should not be interpreted as a causal explanation for grade differences.

python

```
data.groupby("sex")["G3"].mean()
```

```
sex
F     9.966346
M    10.914439
Name: G3, dtype: float64
```

20. Summarize the final-grade distribution

I used describe on G3 to summarize final grades. In the saved output, the mean is approximately 10.42, the median $50th percentile$ is 11, and the observed range is 0 to 20.

python

```
data["G3"].describe()
```

```
count    395.000000
mean      10.415190
std        4.581443
min        0.000000
25%        8.000000
50%       11.000000
75%       14.000000
max       20.000000
Name: G3, dtype: float64
```

21. Select columns for focused analysis

I created data_need, containing seven variables relevant to the questions explored: studytime, failures, absences, schoolsup, age, sex, and G3. Keeping a smaller set makes the later analysis easier to focus on. This step selects columns; it does not remove rows.

python

```
data_need=data[["studytime","failures","absences","schoolsup","age","s
data_need
```

	studytime	failures	absences	schoolsup	age	sex	G3
0	2	0	6	yes	18	F	6
1	2	0	4	no	17	F	6
2	2	3	10	yes	15	F	10
3	3	0	2	no	15	F	15
4	2	0	4	no	16	F	10
...	...	...	...	...	...	...	...
390	2	2	11	no	20	M	9
391	1	0	3	no	17	M	16
392	1	3	3	no	21	M	7
393	1	0	0	no	18	M	10
394	1	0	5	no	19	M	9

395 rows × 7 columns

22. Check the selected dataset's dimensions

I used shape to confirm that data_need has 395 rows and 7 columns. This shows that the column selection kept all student records while reducing the number of variables.

python

```
data_need.shape
```

```
(395, 7)
```

23. Check missing values in the selected columns

I checked each selected column for missing values. The saved output shows zero missing values in all seven columns.

python

```
data_need.isnull().sum()
```

```
studytime    0
failures     0
absences     0
schoolsup    0
age          0
sex          0
G3           0
dtype: int64
```

24. Check duplicate rows in the selected dataset

I counted exact duplicate rows in data_need. The saved output is 30. These rows may share the same values across the seven selected variables even if they are not duplicates across all 33 original columns. They are not automatically errors, because different students can have the same values for these selected attributes

python

```
data_need.duplicated().sum()
```

```
np.int64(30)
```

25. Recheck duplicates in the original dataset

I checked duplicate rows again in the full data DataFrame. The result is 0, confirming that the full 33-column records have no exact duplicates in this check.

python

```
data.duplicated().sum()
```

```
np.int64(0)
```

26. Inspect data types in the selected dataset

I displayed the data types for the seven selected columns. This helps verify which variables are numeric and which are categorical/text fields before plotting or grouping them.

python

```
data_need.dtypes
```

```
studytime    int64
failures     int64
absences     int64
schoolsup      str
age          int64
sex            str
G3           int64
dtype: object
```

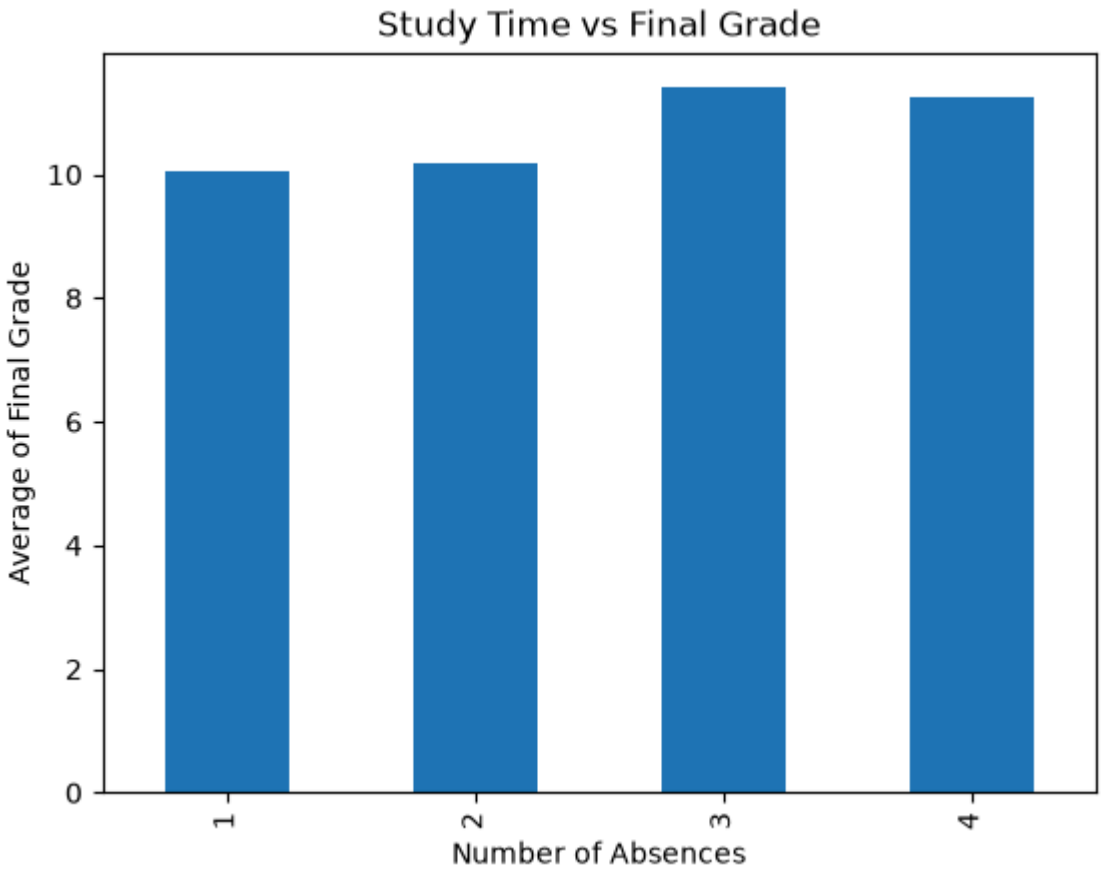
27. Plot study time against average final grade

I grouped data_need by studytime, calculated the mean G3, and plotted the averages as a bar chart. This makes it easier to compare the average final grade across study-time categories.

Chart-label note: The horizontal axis represents study-time categories, not number of absences, so the x-axis label has been corrected below.

python

```
data_need.groupby("studytime")["G3"].mean().plot(kind="bar")
plt.title("Study Time vs Final Grade")
plt.xlabel("Number of Absences")
plt.ylabel("Average of Final Grade")
plt.show()
```



28. Create an overall dashboard of six visualizations

I arranged six plots in a 2-by-3 grid to compare several aspects of student performance:

1.Study time vs final grade: bar plot of average final grades by study-time category.

2.Absences vs final grade: scatter plot showing individual students' absence counts and final grades.

3.Past failures vs final grade: bar plot comparing average final grades across failure categories.

4.Age vs final grade: bar plot comparing average final grades across ages.

5.Sex vs final grade: bar plot comparing average final grades across sex categories.

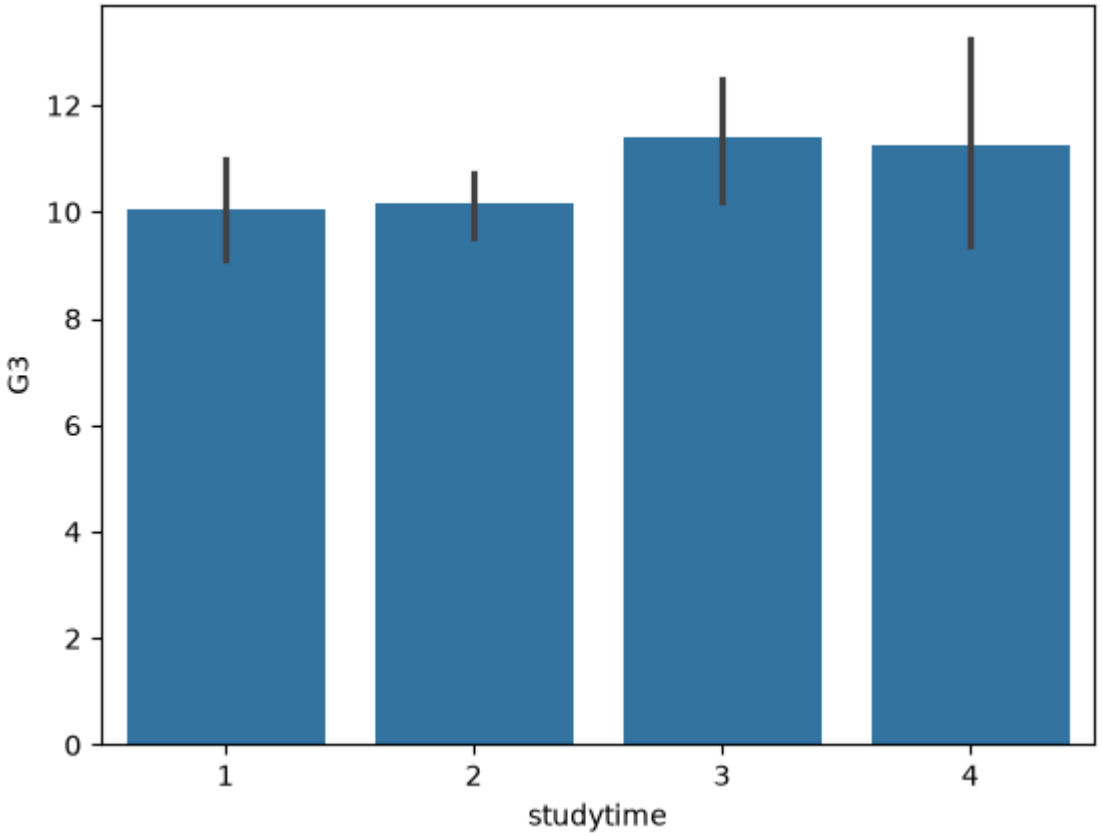
6.Overall final-grade distribution: histogram showing how final grades are distributed.

plt.tight_layout adjusts spacing so the titles and plots fit more clearly, and plt.show displays the dashboard. These visualizations help identify patterns for further discussion; they do not establish causation.

python

```
sns.barplot(data=data,x="studytime",y="G3")
axes[0,0].set_title("Studytime vs Final Grade")
```

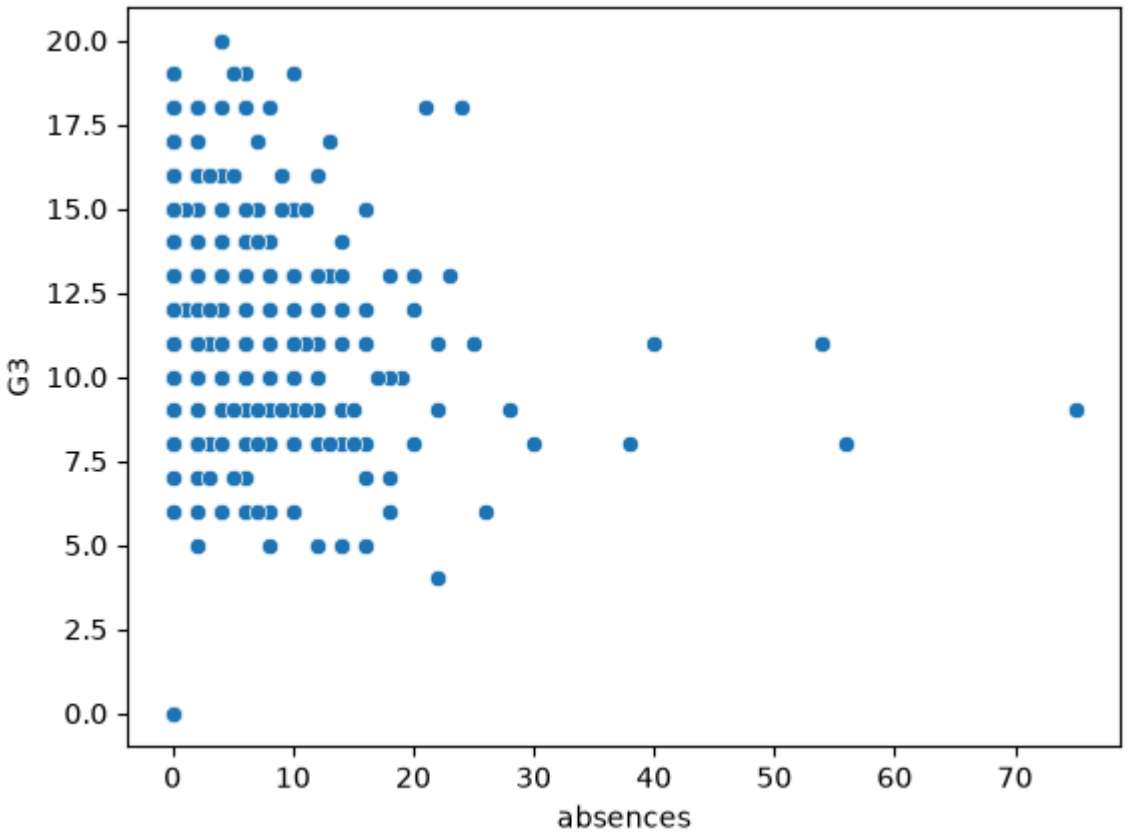
```
Text(0.5, 1.0, 'Studytime vs Final Grade')
```



python

```
sns.scatterplot(data=data,x="absences",y="G3")
axes[0,1].set_title("Absences vs Final Grade")
```

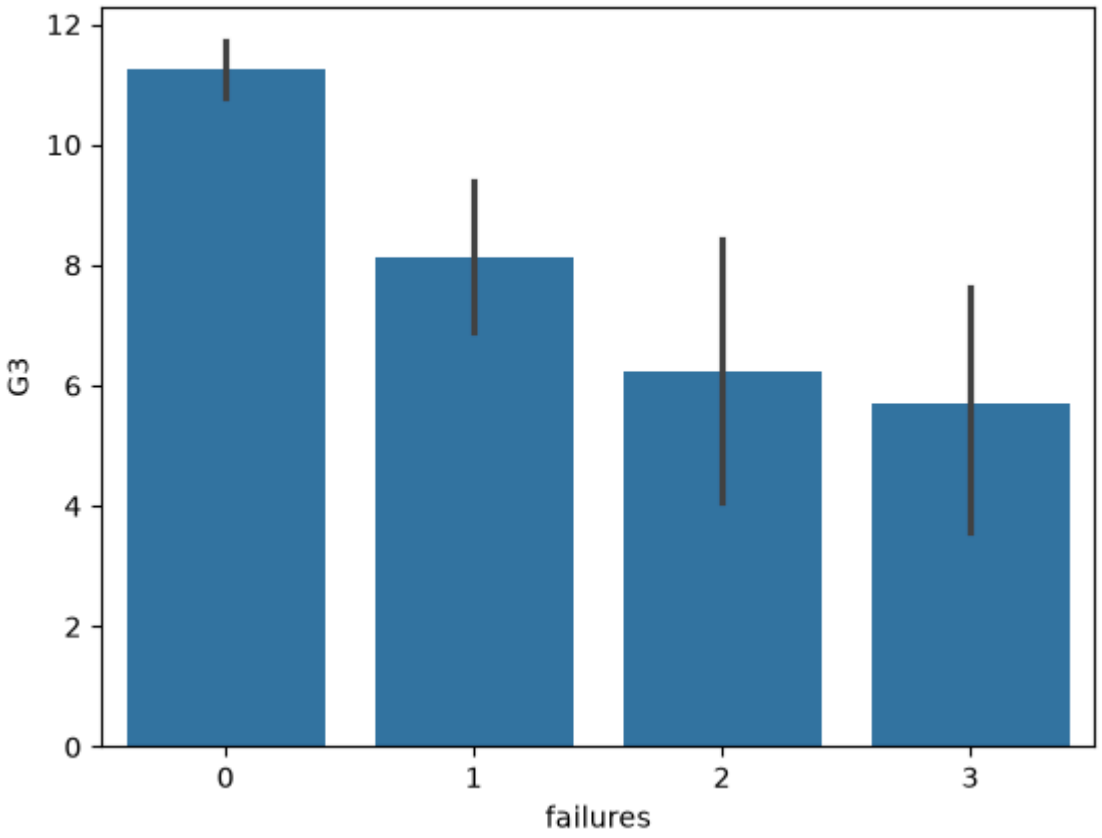
```
Text(0.5, 1.0, 'Absences vs Final Grade')
```



python

```
sns.barplot(data=data,x="failures",y="G3")
axes[0,2].set_title("Past Failures vs Final")
```

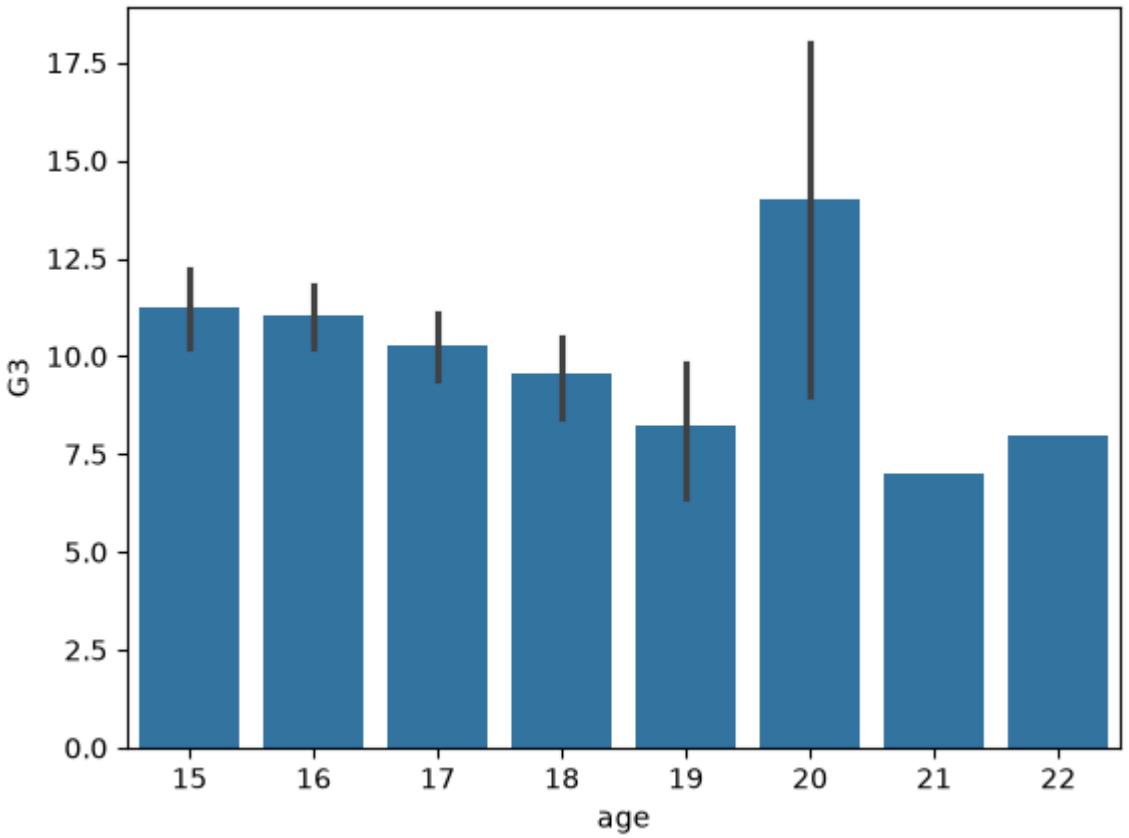
```
Text(0.5, 1.0, 'Past Failures vs Final')
```



python

```
sns.barplot(data=data,x="age",y="G3")
axes[1,0].set_title("Age vs Final Grade")
```

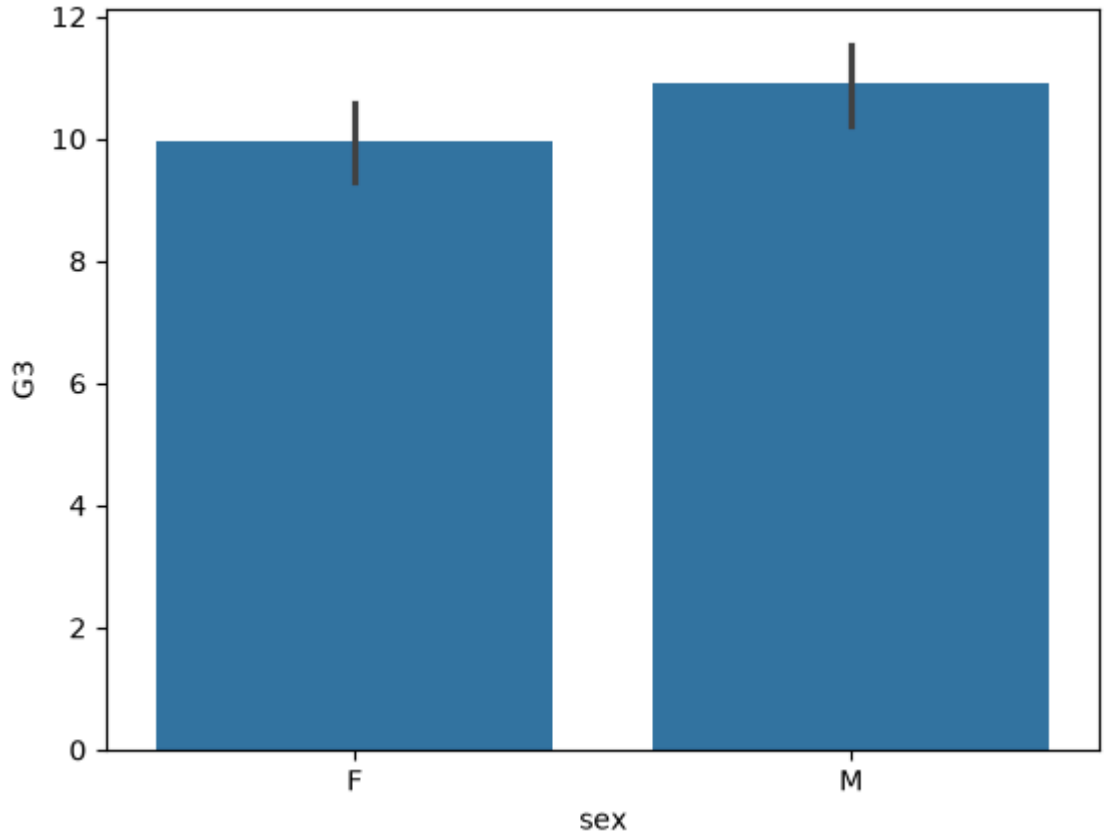
```
Text(0.5, 1.0, 'Age vs Final Grade')
```



python

```
sns.barplot(data=data,x="sex",y="G3")
axes[1,1].set_title("Gender vs Final Grade")
```

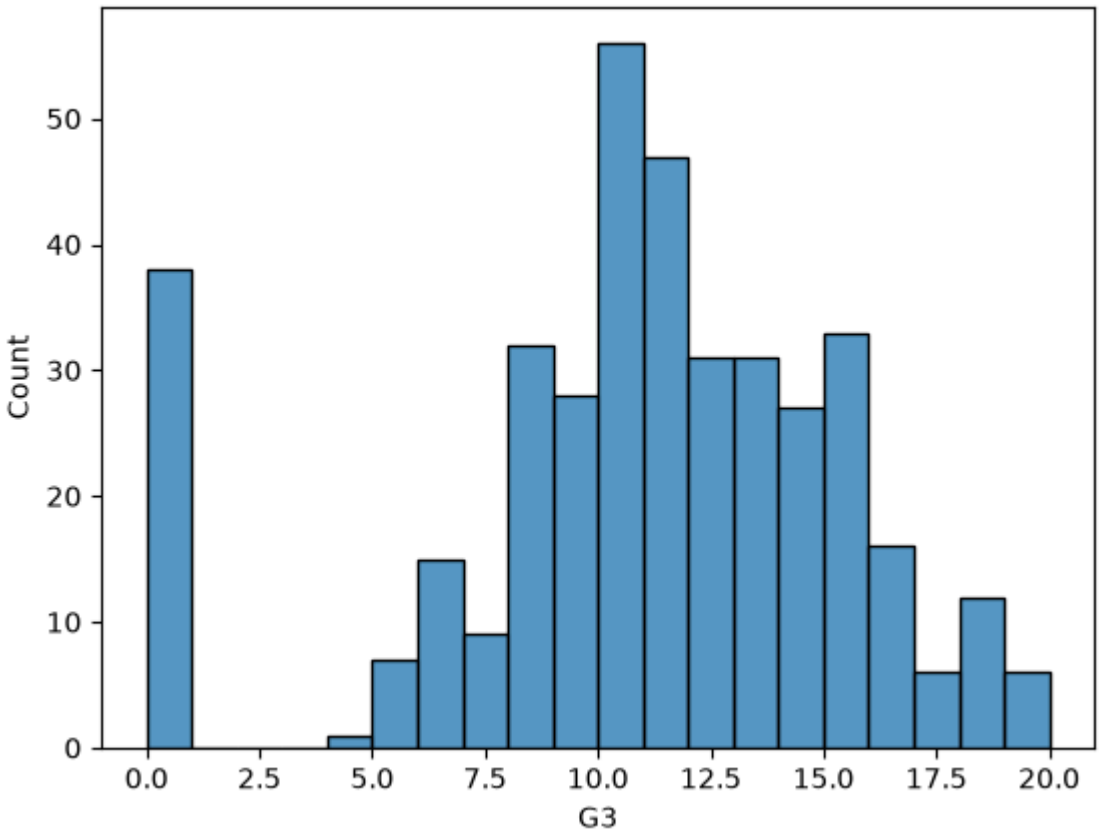
```
Text(0.5, 1.0, 'Gender vs Final Grade')
```



python

```
sns.histplot(data=data,x="G3",bins=20)
axes[1,2].set_title("Overall Final Grade Distributuion")
```

```
Text(0.5, 1.0, 'Overall Final Grade Distributuion')
```



Conclusion

This project used Exploratory Data Analysis to inspect the student mathematics performance dataset, review basic data quality, summarize grade statistics, compare average final grades across selected student-related categories, and visualize the patterns. The findings describe relationships in this dataset; they do not establish cause and effect.